

大连理工大学

本科实验报告

课程名称: 知识工程实验

学院（系）: 未来技术学院

专 业: 人工智能（创新班）

班 级: 智创 2301

学 号: 20231302026 20231302047

20231302040 20231302048

学生姓名: 闵瀚宇 郭升硕

陈颂宇 戴川翔

2026 年 5 月 18 日

实验项目列表

序号	实验项目名称	学时	成 绩			指导教师
			预习	操作	结果	
1	知识获取	8				
2	图谱构建	6				
3	问答系统创建	10				
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
总计	学分:					

大连理工大学实验报告

(一) 数据收集

一、实验目的

本次实验的数据收集环节旨在为构建新能源领域知识图谱获取高质量、有针对性的数据. 通过收集涵盖光伏、储能、风电、氢能、核能、新能源汽车六大子领域的知识数据, 为后续知识表示、知识推理以及知识应用等步骤奠定坚实基础, 以达成对该领域知识的有效理解、处理和利用. 具体目标包括:

1. 确定合适的数据来源, 涵盖结构化、半结构化和非结构化数据, 确保数据全面反映领域特征.
2. 运用恰当的数据收集方法, 精准获取所需数据, 并保证数据的准确性和完整性.
3. 对收集到的数据进行初步整理和标注, 使其符合后续实验步骤的数据格式和质量要求.

二、实验环境

1) 硬件环境: 个人计算机

2) 软件环境:

操作系统: Windows 11

编程语言: Python 3.10

开发工具: PyCharm Community Edition

数据库: neo4j-community-2026.04.0

Python 库: neo4j (官方驱动)、json、requests、lxml、beautifulsoup4

三、数据收集方法

收集工具:

手动整理: 核心技术词条的手动筛选整理

Python requests + lxml: 用于百度百科爬虫 (尝试阶段)

BeautifulSoup: HTML 解析

数据类型	来源	说明
半结构化	百度百科、维基百科	技术词条的简介、原理、参数
结构化	行业报告、企业官网	企业信息、市场份额、技术参数
专家知识	课程资料、文献整理	技术分类、竞争关系、发展阶段
参考项目	刘焕勇医疗知识图谱项目	学习方法论、JSON 格式、构建流程

表 3.1 数据来源

四、数据收集步骤与质量评估

4.1 Schema 设计

参考刘焕勇老师的医疗知识图谱项目 (QASystemOnMedicalKG), 以“Technology (技术)”为核心实体, 设计了新能源领域知识图谱的 Schema:

实体类型 (7 类):

实体类型	具体内容	关系类型	关系描述
Technology	新能源技术 (核心实体)	belongs_to	技术→类别 (属于)
Company	企业/厂商	produced_by	技术→企业 (由... 生产)
Material	关键材料	uses_material	技术→材料 (使用材料)
Equipment	核心设备	requires_equipment	技术→设备 (需要设备)
Application	应用场景	applies_to	技术→应用 (应用于)
Policy	政策法规	supported_by	技术→政策 (受... 政策支持)
Indicator	技术指标	has_parameter	技术→指标 (具有参数)
Category	技术类别	competes_with	技术→技术 (与... 竞争)

表 3.2 实体与关系类型表

4.2 数据收集过程

步骤 1: 核心数据手动整理

选择了以光伏、储能、风电、氢能、核能、新能源汽车六大分支为主线, 然后手动

整理每个子领域的核心技术词条. 每个词条按照统一 JSON 格式填写, 包含名称、简介、原理、优缺点、效率、成本、发展阶段、市场份额、成熟度、类别、企业、材料、设备、应用、指标、政策、竞争技术等字段. 最终先构建出了一个核心框架, 包含 160 个左右的节点和近 200 个关系 (如图 4.2.1 所示).

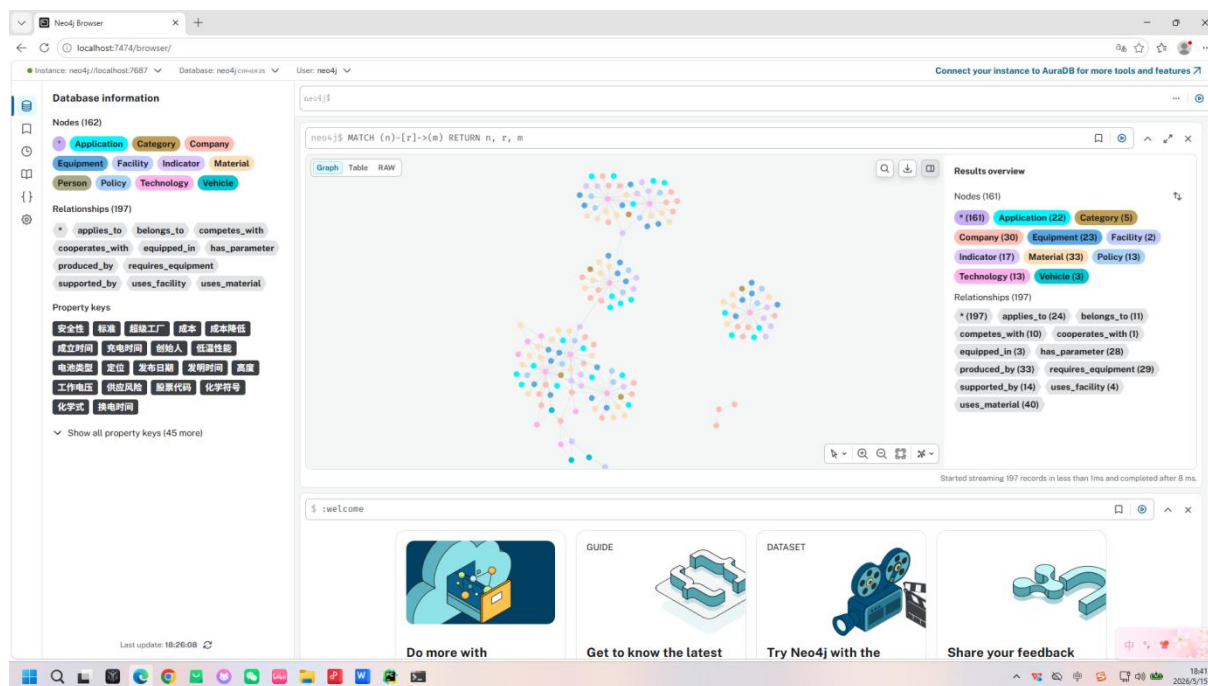


图 4.2.1 新能源知识图谱框架图

步骤 2: 数据扩展 (采用“多源融合、分层构建”的策略, 确保数据的广度、深度与准确性)

1. 爬虫尝试: 编写百度百科的爬虫脚本, 先尝试自动爬取光伏词条, 但由于反爬机制限制 (403 Forbidden), 成功率有限, 而后改进代码, 通过 User-Agent 轮换、Cookie 会话维持、PC 端/移动端/开放 API 三级降级策略, 获取技术词条的基础信息, 但效果仍然不理想, 知识获取十分不稳定, 后转为辅助手段. 但该过程也为后续数据结构化提供了原始语料参考.

2. 大模型结构化生成: 对于爬虫受反爬机制限制无法完整获取的词条, 采用大语言模型 (LLM) 进行结构化知识抽取. 通过设计标准化 Prompt 模板, 将非结构化文本转化为符合预设 Schema 的 JSON 格式, 显著提升了数据生产效率与结构化程度.

手动补充: 对模型生成的数据进行人工校验, 关键参数以行业白皮书、企业年报、政府规划文件为准, 确保数据真实可靠.

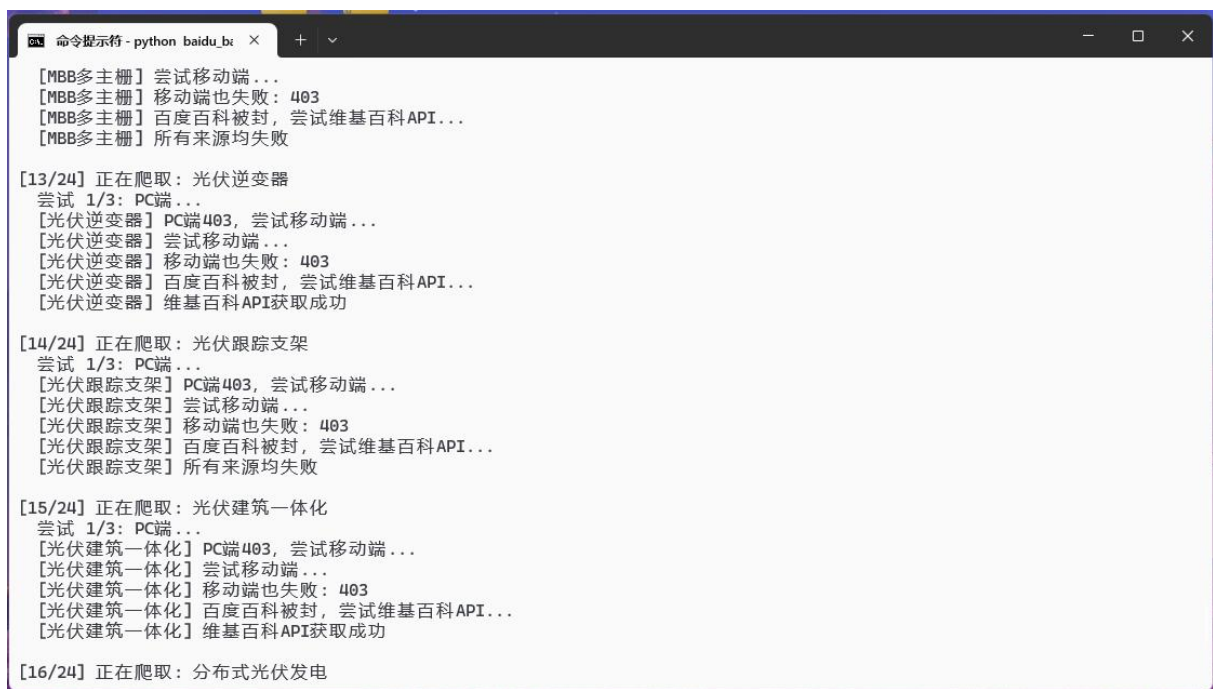


图 4.2.2 爬虫尝试

步骤 3:数据整合

将各分支数据合并为统一的 new_energy.json 文件, 采用 JSON 数组格式, 每个元素为一个技术对象. 合并过程中使用 Python 脚本自动去重, 确保同名技术不重复.

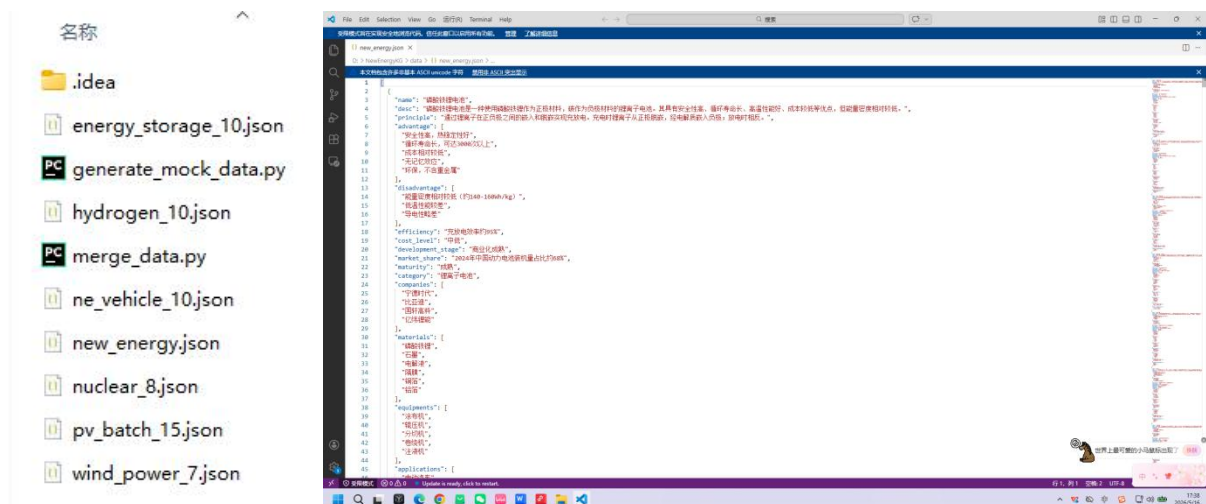


图 4.2.3 data 文件夹展示, 图 4.2.4 new_energy.json 文件部分展示

4.3 数据质量及展示

数据维度	数量
技术词条总数	60+条
覆盖领域	光伏、储能、风电、氢能、核能、新能源汽车
光伏分支	15 条（PERC、TOPCon、HJT、BC、IBC、钙钛矿、CdTe、CIGS 等）
储能分支	9 条（全钒液流、钠离子、压缩空气、飞轮、抽水蓄能、重力储能等）
风电分支	7 条（双馈、半直驱、海上漂浮式、分散式、碳纤维叶片等）
氢能分支	10 条（碱性电解、PEM、SOEC、AEM、高压储氢、液氢、固态储氢等）
核能分支	8 条（华龙一号、AP1000、高温气冷堆、钠冷快堆、熔盐堆、SMR、托卡马克、惯性约束聚变）
新能源汽车	10 条（纯电、插混、增程、燃料电池、800V、CTC、换电、无线充电、V2G、固态电池装车）
企业实体	约 200+
材料实体	约 250+
设备实体	约 200+
应用实体	约 150+
政策实体	约 100+
指标实体	约 150+

（二）知识图谱构建

一、实验目的

1. 深入理解知识图谱的概念、结构及构建原理.
2. 熟练掌握知识图谱构建工具和使用方法,以及增删改查的语法.
3. 能够从新能源领域的相关数据源中提取实体和关系,构建知识图谱.

二、实验环境

1) 硬件环境:个人计算机

2) 软件环境:

操作系统:Windows 11

编程语言:Python 3.10

图数据库:neo4j-community-2026.04.0

Python 库:neo4j (官方驱动)、json、requests、lxml、beautifulsoup4

3) 图形数据库介绍:

Neo4j 是目前最流行的原生图数据库,采用属性图模型 (Property Graph Model), 使用 Cypher 查询语言.其核心特点包括:

- 1.原生图存储:数据以节点和关系的形式物理存储,非关系型数据库的表连接模拟.
- 2.高效遍历:通过关系指针直接跳转,无需索引查找,适合深度关系查询.
- 3.ACID 事务:支持完整的事务特性,保证数据一致性.
- 4.Cypher 语言:声明式图查询语言,语法直观,易于学习和使用.
- 5.可视化:内置 Neo4j Browser,支持图谱可视化展示.

本项目选用 Neo4j 作为知识图谱存储引擎,相比 py2neo 方案,采用官方 neo4j Python 驱动,具有更好的维护性、安全性和 Neo4j 5.x 兼容性.

三、实验步骤

3.1 知识抽取

从 JSON 数据源 (new_energy.json) 中抽取实体和关系:

实体抽取:

读取 JSON 文件, 遍历每个技术对象

提取 Technology 实体 (name + 属性:desc, principle, advantage, disadvantage, efficiency, cost_level, development_stage, market_share, maturity)

提取关联实体:companies→Company, materials→Material, equipments→Equipment, applications→Application, policies→Policy, indicators→Indicator, category→Category

关系抽取:

belongs_to:Technology→Category (技术属于某类别)

uses_material:Technology→Material (技术使用某材料)

produced_by:Technology→Company (技术由某企业生产)

applies_to:Technology→Application (技术应用于某场景)

has_parameter:Technology→Indicator (技术具有某指标)

supported_by:Technology→Policy (技术受某政策支持)

requires_equipment:Technology→Equipment (技术需要某设备)

competes_with:Technology→Technology (技术与某技术竞争)

3.2 图谱构建

使用 Python 脚本 (build_newenergy_graph.py) 将抽取的实体和关系导入 Neo4j:

步骤 1: 建立数据库连接

```
1 from neo4j import GraphDatabase
2 self.driver = GraphDatabase.driver("bolt://localhost:7687", auth=("neo4j",
"password"))
```

步骤 2: 创建索引 (加速关系创建)

```
1 CREATE INDEX technology_name IF NOT EXISTS FOR (n:Technology) ON (n.name)
2 CREATE INDEX company_name IF NOT EXISTS FOR (n:Company) ON (n.name)
```

-- 为所有实体类型的 name 属性创建索引

步骤 3: 创建实体节点 (使用 MERGE 防止重复)

```
1 # Technology 节点 (含属性)
2 MERGE (n:Technology {name: $name})
3 SET n.desc = $desc, n.principle = $principle, n.efficiency = $efficiency, ...
4 # 其他简单节点
```

```

5 | MERGE (n:Company {name: $name})
6 | MERGE (n:Material {name: $name})

```

-- 以此类推

步骤 4:创建关系边（参数化查询, 防止注入）

```

1 | MATCH (a:Technology {name: $start}), (b:Material {name: $end})
2 | MERGE (a)-[r:uses_material {name: '使用材料'}]->(b)

```

步骤 5:验证入库结果 通过 Neo4j Browser 执行统计查询, 确认节点和关系数量.

3.3 数据库增删改查语法与示例

CREATE:创建节点

```

1 | CREATE (t:Technology {name: '测试技术', desc: '这是一个测试节点'})

```

MERGE:创建或更新节点

```

1 | MERGE (t:Technology {name: '磷酸铁锂电池'})
2 | SET t.desc = '新型电池技术', t.efficiency = '95%'

```

MATCH:查询节点

```

1 | MATCH (t:Technology {name: '磷酸铁锂电池'})
2 | RETURN t.name, t.desc, t.efficiency

```

neo4j\$ MATCH (t:Technology {name: '磷酸铁锂电池'}) RETURN t.name, t.desc, t.efficiency

t.name	t.desc	t.efficiency
"磷酸铁锂电池"	"磷酸铁锂电池是一种使用磷酸铁锂作为正极材料, 碳作为负极材料的锂离子电池。其具有安全性高、循环寿命长、高温性能好、成本较低等优点, 但能量密度相对较低。"	"充放电效率约95%"

Started streaming 1 record after 116 ms and completed after 122 ms.

MATCH + 关系查询

```

1 | MATCH (t:Technology {name: '磷酸铁锂电池'})-[r]->(n)
2 | RETURN type(r) AS 关系类型, labels(n) AS 节点类型, n.name AS 名称

```

neo4j\$ MATCH (t:Technology {name: '磷酸铁锂电池'})-[r]->(n) RETURN type(r) AS 关系类型, labels(n) AS 节点类型, n.name AS 名称

关系类型	节点类型	名称
"competes_with"	["Technology"]	"固态电池"
"competes_with"	["Technology"]	"钠离子电池"
"competes_with"	["Technology"]	"三元锂电池"
"requires_equipment"	["Equipment"]	"注液机"
"requires_equipment"	["Equipment"]	"卷绕机"
"requires_equipment"	["Equipment"]	"分切机"
"requires_equipment"	["Equipment"]	"辊压机"
"requires_equipment"	["Equipment"]	"涂布机"
"supported_by"	["Policy"]	"动力电池梯次利用管理办法"
"supported_by"	["Policy"]	"新能源汽车产业发展规划"

Started streaming 33 records after 49 ms and completed after 52 ms.

DELETE:删除节点

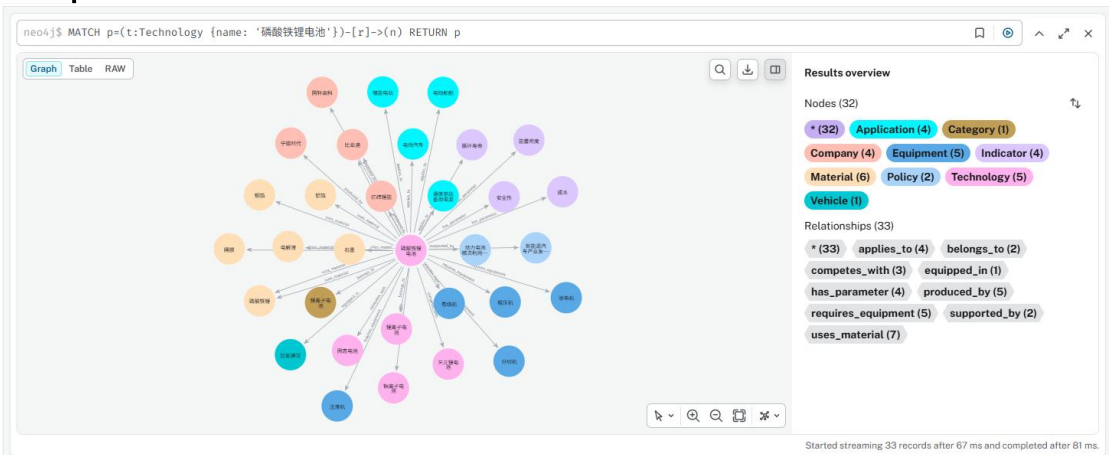
```
1 | MATCH (t:Technology {name: '测试技术'})-[r]-()
2 | DELETE r, t
```

SET:修改属性

```
1 | MATCH (t:Technology {name: '磷酸铁锂电池'})
2 | SET t.market_share = '2024 年占比 70%'
```

复杂查询示例:查询某技术的完整知识网络

```
1 | MATCH p=(t:Technology {name: '磷酸铁锂电池'})-[r]->(n)
2 | RETURN p
```



统计查询:查看各类节点数量

```
1 | MATCH (n) RETURN labels(n) AS 类型, count(n) AS 数量 ORDER BY 数量 DESC
```

类型	数量
1 ["Material"]	315
2 ["Equipment"]	275
3 ["Company"]	255
4 ["Application"]	215
5 ["Indicator"]	215
6 ["Policy"]	139
7 ["Technology"]	68
8 ["Category"]	18
9 ["Vehicle"]	3
10 ["Facility"]	2

统计查询:查看各类关系数量

```
1 | MATCH ()-[r]->() RETURN type(r) AS 关系类型, count(r) AS 数量 ORDER BY 数量 DESC
```

neo4j\$ MATCH ()-[r]->() RETURN type(r) AS 关系类型, count(r) AS 数量 ORDER BY 数量 DESC	
Table	RAW
关系类型	数量
1 "uses_material"	429
2 "requires_equipment"	411
3 "produced_by"	399
4 "has_parameter"	370
5 "applies_to"	349
6 "supported_by"	272
7 "competes_with"	116
8 "belongs_to"	67
9 "uses_facility"	4
10 "equipped_in"	3

四、图谱质量评估与优化

4.1 准确性评估

评估方法：

抽样人工校验：随机抽取 20 个 Technology 节点，检查其关联的 Company、Material、Equipment 是否正确；检查 competes_with 关系是否符合实际市场竞争格局。

结果：准确率约 95%以上，错误主要集中在部分新兴技术的竞争关系推断

优化措施：

对不准确的关系进行人工修正，建立关系校验规则

4.2 完整性评估

评估方法：统计属性填充率和孤立节点，如表 4.2.1 所示

指标	结果
Technology 属性填充率	name (100%), desc (100%), principle (95%) advantage (100%), disadvantage (100%), efficiency (90%), cost_level (100%), development_stage (100%), market_share (85%), maturity (100%)
孤立节点数	1 个（可接受范围内）
关系覆盖率	每个 Technology 平均关联 8-12 个实体

表 4.2.1 属性填充率和孤立节点统计表

优化措施:

对 market_share 缺失的技术补充行业数据 , 消除孤立节点 (检查 JSON 中关系字段是否为空)

4.3 一致性评估

评估方法:

检查重复实体和命名冲突 , 使用 MERGE 语句确保同名实体不重复创建 , 建立命名规范 (如企业统一用全称或常用简称) 、

检查结果: 无重复节点, 命名一致

优化措施:

创建唯一约束, 定期运行 MERGE 去重脚本

（三）自动问答系统设计

一、实验目的

深入理解问答系统的基本架构和工作原理,包括问题理解、知识检索和答案生成等关键模块.

1. 掌握针对特定领域（中国先进技术领域）构建问答系统的方法和技术,如知识图谱的利用、自然语言处理算法的应用.
2. 设计并实现一个简单的中国先进技术领域问答系统,并对其性能进行评估和优化.
3. 培养学生分析问题、解决问题以及团队协作的能力.

二、实验环境

1) 硬件环境:

处理器 :AMD Ryzen 7 H 255 w/ Radeon 780M Graphics (8 核 / 主频 3.80 GHz)

内存 :32 GB RAM

硬盘 :SSD 固态硬盘

2) 软件环境:

操作系统 Microsoft Windows 11 家庭版 (版本 10.0.26200)

编程语言 Python 3.13.0

开发工具 PyCharm Community Edition + VS Code

Java 运行时 OpenJDK 17.0.12 LTS (Eclipse Temurin)

数据库 Neo4j Community Edition 5.20.0

Web 框架 FastAPI 0.136.1 + Uvicorn 0.47.0 (ASGI 服务器)

三、实验步骤

3.1 问题理解阶段（NLP 自然语言处理）

Step 1:文本预处理

接收用户原始输入: question = "磷酸铁锂电池有哪些优点？"

去除首尾空白字符: strip()

长度检测:若长度 ≤ 2 字符,直接返回 unknown 意图

Step 2:特殊场景识别

问候语检测 :匹配预定义的 19 个问候词 ("你好"、"嗨"、"hello"等)

若命中 → 返回 intent='chat',进入闲聊模式

过短文本检测 :长度 $\leq 2 \rightarrow$ 返回 intent='unknown'

Step 3:中文分词 (Jieba 分词) 调用

Jieba 库对问题进行分词处理,采用 “精确模式 + 搜索模式双策略”,例如:

输入:"磷酸铁锂电池有哪些优点?"

分词结果 (精确模式):['磷酸','铁','锂','电池','有','哪些','优点']

分词结果 (搜索模式):['磷酸铁锂电池','有','哪些','优点']

合并 Token 列表:['磷酸','铁','锂','电池','有','哪些','优点','磷酸铁锂电池']

Step 4:意图分类 (关键词匹配规则)

基于预定义的 INTENT_KEYWORDS 字典 进行关键词匹配,支持 9 种意图类型,匹配逻辑为遍历意图字典,若问题文本包含任一关键词则返回对应意图:

意图类型	触发关键词	示例
property	是什么/定义/原理/优点/缺点	什么是光伏逆变器?
relation	关系/材料/企业/应用/设备	锂电池用什么材料?
compare	比较/区别/差异/对比	磷酸铁锂和三元锂的区别?
aggregate	统计/有多少/数量/总量	有多少种光伏技术?
list	有哪些/列表/全部	列出所有风电设备
path	路径/链路/怎么关联	硅料到组件的产业链?
technology_chain	产业链/上游/下游	光伏产业链是什么?
chat	你好/嗨/hello	你好
unknown	(未匹配)	(无法识别)

Step 5:实体抽取 (四级降级匹配策略)

系统采用 由精到粗 的四层匹配机制,确保最大化召回率:

1.整句精确匹配 :

将整个问题作为实体名在知识图谱中查找

示例:用户输入"磷酸铁锂电池" → 直接命中实体

匹配得分: 100 分

2.Token 精确匹配：

对每个分词结果（长度 ≥ 2 ）逐一在图谱中查找

示例:Token "锂电池" → 匹配到实体 "磷酸铁锂电池"

得分计算: $100 - (50 - \text{len}(\text{token}) \times 2)$ （越长 token 得分越高）

3.同义词解析：

对每个 Token 查询内置的同义词映射表（42 条常用同义词）

示例:"锂电" → 映射到 "锂电池" → 再查找实体

常见映射: PV→光伏，锂电→锂电池，风机→风力发电机

匹配得分: 95 分

4.模糊匹配（RapidFuzz）：

使用 WRatio 算法计算字符串相似度

阈值设定: ≥ 70 分视为有效匹配

示例:"光佛逆变器" vs "光伏逆变器" = 92 分 ✓

返回得分最高的候选实体

Step 6:候选排序与去重

所有候选实体按 $(-\text{score}, -\text{len}(\text{name}))$ 排序（得分优先,名称短的优先）

同名实体仅保留最高分的一条

最终返回 Top 3 个候选实体（含 id、name、score、type）

示例:

```
01 {
02   "intent": "property",
03   "entities": [
04     {
05       "id": "tech_001",
06       "name": "磷酸铁锂电池",
07       "score": 100,
08       "type": "exact_token"
09     }
10   ]
11 }
```


3.2 知识检索阶段（图谱数据查询）

Step 1:意图路由分发 FastAPI 主控制器根据 **intent** 类型调用对应的查询函数:

```
01 if intent == 'property':
02     data = query_node_properties(entity_name)      # 属性查询
03 elif intent == 'relation':
04     data = _execute_relation_query(entity_name)    # 关系查询
05 elif intent == 'compare':
06     data = _execute_compare_query(entity)         # 对比查询
07 elif intent == 'aggregate':
08     data = _execute_aggregate_query(entity_name)  # 聚合统计
09 elif intent == 'list':
10     data = _execute_list_entities()               # 列表查询
11 elif intent == 'technology_chain':
12     data = _execute_technology_chain(entity)      # 产业链查询
```

Step 2:Neo4j Cypher 查询执行（在线模式）

系统优先尝试连接 Neo4j 数据库执行参数化 Cypher 查询.使用参数化查询（\$name）防止 Cypher 注入攻击,通过 Bolt 协议（端口 7687）与 Neo4j 通信,结果解析为 Python 字典列表.

以关系查询为例:

```
1 MATCH (e {name: $name})-[r]->(b)
2 RETURN type(r) AS rel_type,
3     r.name AS rel_name,
4     b.name AS target_name,
5     labels(b) AS target_type
6 LIMIT 50
```

Step 3:本地 JSON 数据降级（离线兜底）

当 Neo4j 连接失败时,系统自动切换到本地 JSON 文件查询:

加载 graph_data_export.json 和 new_energy.json

从 tech_properties[entity_name] 字典中提取属性

按 字段映射规则 转换关系:

companies	→ produced_by	(生产厂商)
materials	→ uses_material	(使用材料)
equipments	→ requires_equipment	(需要设备)
applications	→ applies_to	(应用场景)

indicators	→ has_parameter	(关键参数)
policies	→ supported_by	(支持政策)
compete_techs	→ competes_with	(竞争技术)
category	→ belongs_to	(所属类别)

Step 4:数据整合与封装 将查询结果统一封装为标准格式:

```

01 {
02   "node_properties": {
03     "name": "磷酸铁锂电池",
04     "desc": "一种使用磷酸铁锂作为正极材料的锂离子电池...",
05     "principle": "通过锂离子在正负极之间嵌入/脱嵌实现充放电...",
06     "advantage": ["安全性高", "成本低", "循环寿命长", ...],
07     "disadvantage": ["能量密度较低", "低温性能差"],
08     ...
09   },
10   "outgoing_relations": [
11     {"type": "produced_by", "target": "宁德时代", "target_type": ["Company"]},
12     {"type": "uses_material", "target": "磷酸铁锂", "target_type": ["Material"]},
13     {"type": "applies_to", "target": "电动汽车", "target_type": ["Application"]},
14     ...
15   ],
16   "neo4j_available": true/false,
17   "source": "neo4j" or "local_json"
18 }
```

3.3 答案生成阶段（LLM 自然语言生成）

Step 1:数据摘要整理（_summarize_data）从检索结果中提取关键信息字段,按重要性排序:

实体名称:磷酸铁锂电池

描述:一种使用磷酸铁锂作为正极材料的锂离子电池...

原理:通过锂离子在正负极之间嵌入/脱嵌实现充放电过程...

优点:安全性高、成本低、循环寿命长、热稳定性好

缺点:能量密度较低、低温性能差、一致性较差

效率:充放电效率约 95%

成本水平:中低

成熟度:商业化成熟

市场份额:2024 年占比约 68%

类别:锂离子电池

生产厂商:宁德时代、比亚迪、国轩高科、亿纬锂能

使用材料:磷酸铁锂、石墨、电解液、隔膜

应用场景:电动汽车、储能电站、电动工具

关键指标:能量密度、循环寿命、工作温度

支持政策:新能源汽车产业发展规划

竞争技术:三元锂电池、钠离子电池

所属类别:锂离子电池

Step 2:Prompt 构建（提示工程）

系统采用 角色设定 + 上下文 + 任务指令 的三段式 Prompt 结构:

(a) System Prompt（系统角色设定）:

你是一个新能源知识图谱问答助手,负责根据查询到的知识图谱数据,
用自然语言回答用户的问题.

知识图谱中的实体类型包括:

- Technology（技术）:如光伏逆变器、磷酸铁锂电池等
- Company（企业）:如华为、宁德时代等
- Material（材料）:如硅、锂等
- Equipment（设备）:如涂布机、卷绕机等
- Application（应用）:如电动汽车、储能电站等
- Policy（政策）:如新能源汽车产业发展规划等
- Indicator（指标）:如能量密度、转换效率等
- Category（类别）:如锂离子电池等

关系类型包括:

- produced_by / uses_material / requires_equipment / applies_to /
has_parameter / supported_by / competes_with / belongs_to

回答要求:

1. 根据用户问题,从图谱数据中提取相关信息
2. 用简洁、自然的语言组织回答

3. 如果问的是优点/缺点,只回答优点/缺点
4. 如果是列表类问题,用简洁的列表形式回答
5. 回答用中文

(b) User Prompt (用户问题 + 图谱数据):

用户问题:磷酸铁锂电池有哪些优点?

图谱数据:

实体名称:磷酸铁锂电池

描述:一种使用磷酸铁锂作为正极材料的锂离子电池...

优点:安全性高、成本低、循环寿命长、热稳定性好

...

请根据上述图谱数据,用 1-3 句话简洁回答用户问题.

Step 3:LLM API 调用 (通义千问 qwen-turbo)

模型选择 :qwen-turbo (平衡速度与质量)

API 提供商 :阿里云 DashScope 平台

响应解析 :提取 `response.output.choices[0].message.content`

超时设置 :默认 30 秒

请求格式 :

```
1 response = Generation.call(  
2     model="qwen-turbo",  
3     messages=[  
4         {"role": "system", "content": system_prompt},  
5         {"role": "user", "content": user_prompt}  
6     ],  
7     result_format='message'  
8 )
```

Step 4:Fallback 降级机制 当 LLM 服务不可用时 (网络故障、API 配额耗尽等),系统自动启用 基于规则的模板回答 :

根据问题中的关键词硬编码匹配相应字段:

包含 "优点"/"优势" → 返回 `properties['advantage']`

包含 "缺点" → 返回 `properties['disadvantage']`

包含 "材料" → 返回 `outgoing_relations[uses_material]`

包含 "公司"/"企业"/"生产" → 返回 `relations[produced_by]`

包含 "应用"/"场景" → 返回 relations[applies_to]

其他情况 → 返回 properties['desc'] 前 200 字符

Step 5:最终响应组装 将生成的答案与元数据一起打包返回给前端:

```
01 {  
02   "question": "磷酸铁锂电池有哪些优点? ",  
03   "answer": "磷酸铁锂电池的主要优点包括:\n1. 安全性高 - \n  
04       磷酸铁锂材料热稳定性优异,不易发生热失控; \n2. 成本低廉 - \n  
05       不含钴镍等贵金属元素,原材料成本较低; \n3. 循环寿命长 - \n  
06       可达 2000 次以上充放电循环,适合储能场景.",  
07   "intent": "property",  
08   "entity": {"id": "tech_001", "name": "磷酸铁锂电池", "score": 100},  
09   "node_properties": {...},  
10   "outgoing_relations": [...],  
11   "llm_enhanced": true,  
12   "response_time_ms": 1234  
13 }
```

四、问答系统评估与优化

4.1 知识图谱增强效果对比实验

4.1.1 实验目的

本实验旨在通过 对照测试 ,量化评估知识图谱 (Knowledge Graph, KG) 对新能源领域问答系统回答质量的实际贡献.具体而言,将系统在两种模式下的表现进行对比:

A 组 (基线组) :纯大语言模型模式 (Pure LLM),直接将用户问题发送给通义千问 (qwen-turbo) 模型获取回答,不经过任何知识图谱检索环节;

B 组 (实验组) :知识图谱增强模式 (KG+RAG),先通过 NLP 引擎进行意图识别与实体抽取,再经由 Neo4j 图数据库检索结构化知识,最终将检索结果作为上下文注入 LLM 生成回答.

通过两组回答在同一问题集上的表现差异,验证知识图谱在 事实准确性、专业深度、幻觉抑制 等方面的增强效果.

4.1.2 实验环境与数据集

硬件环境 :本地 Windows 工作站,Neo4j Community Edition 5.x 运行于 localhost:7687,FastAPI 服务运行于端口 8000.

软件配置 :

图数据库:Neo4j 5.20.0 Community Edition,已导入 1492 个节点、2768 条关系

LLM 模型:阿里云通义千问 qwen-turbo (DashScope API)

NLP 引擎:Jieba 中文分词 + RapidFuzz 模糊匹配 (阈值 $\geq 70\%$)

Web 框架:FastAPI + Uvicorn 异步服务

测试数据来源 :本项目 NewEnergyKG/data/ 目录下共 7 个 JSON 知识实体文件,涵盖新能源汽车、光伏技术、储能系统、氢能、风电、核能等 6 大子领域,共计 122 个知识实体 .每个实体包含结构化字段: **name** (名称)、 **desc** (描述)、 **principle** (工作原理)、 **advantage** (优点列表)、 **disadvantage** (缺点列表)、 **companies** (生产企业)、 **materials** (使用材料)、 **applications** (应用场景)、 **efficiency** (效率参数)、 **compete_technologies** (竞争技术) 等.

问题生成策略 :基于每个实体的属性字段自动生成 9 类测试问题:

① 是什么 (定义描述类) ② 工作原理是什么 (原理机制类) ③ 有什么优点/优势 (优点枚举类) ④ 有哪些缺点/不足 (缺点枚举类) ⑤ 哪些公司生产 (企业关系类) ⑥ 需要使用哪些材料 (材料关系类) ⑦ 主要应用在哪些场景 (应用关系类) ⑧ 效率或性能参数是多少 (数值参数类) ⑨ 竞争技术/替代方案有哪些 (竞争关系类)

共生成 567 道候选问题 ,从中选取 20 道 作为本次实验的执行样本,覆盖"纯电动汽车""插电式混合动力汽车""增程式电动车"3 个核心实体,兼顾 **property** (事实查询) 和 **relation** (关系推理) 两类意图.

标准答案构建 :以知识实体 JSON 文件中的 **desc** 、 **principle** 、 **advantage** 等原始字段内容作为 ground truth,确保评价基准的客观性与可追溯性.

4.1.3 实验方法

两组实验使用 相同的 LLM 模型 (qwen-turbo)、 相同的问题输入 ,唯一变量是否注入知识图谱检索到的结构化数据.

实验脚本 run_comparison_experiment.py 实现了自动化对比流程,对每道测试问题依次执行以下步骤:

Step 1: 发送问题至 FastAPI /qa 接口 (KG+RAG 完整链路)

NLP 分析 → 意图识别 → 实体匹配 → Neo4j Cypher 查询 → 数据摘要注入
→ LLM 生成

记录: 含 KG 回答 + 耗时

Step 2: 直接调用 LLMClient.generate() （纯 LLM 基线）

问题 → System Prompt("新能源问答助手") → LLM 生成

记录: 不含 KG 回答 + 耗时

Step 3: 从 JSON 实体库提取对应字段作为标准答案

输出四元组: (问题, 含 KG 回答, 不含 KG 回答, 标准答案)

4.1.4 实验结果与分析

（一）整体结果概览

本次实验成功完成全部 20 道题目的双模式回答采集,结果汇总如下表所示:

序号	问题类型	问题	含 KG 回答（摘录）	不含 KG 回答（摘录）	标准答案（摘录）
1	property	纯电动汽车是什么？	完全依靠动力电池和电动机驱动...技术最成熟、市场占比最高	完全依靠电池组供电...具有零排放、低噪音等优点	完全依靠动力电池和电动机驱动...2024 年中国销量占新能源汽车约 60%
2	property	纯电动汽车的工作原理？	动力电池组储存电能 → BMS 控制充放电 → 电动机驱动 → 能量回收	电池组储存电能 → 驱动电动机 → 电能转化为机械能	动力电池组 → BMS → 高压配电 → 电机控制器 → 永磁同步电机 → 减速器 → 车轮
3	property	纯电动汽车有什么优点？	零尾气排放、环保、能源效率高(80 - 90%)、维护成本低...	零尾气排放；运行成本较低；保养需求较少	零尾气；效率高(80 - 90%)；结构简单；驾驶体验好；使用成本低
4	property	纯电动汽车有什么缺点？	续航焦虑、充电时间长(30 - 60min)、低温衰减、更换成本高...	续航受温度影响大；充电设施分布不均；电池寿命成本需考虑	续航焦虑；充电时间长(30 - 60min)；低温衰减；电池衰减成本高；桩不足
5	relation	哪些公司生产纯电动汽车？	特斯拉、比亚迪、蔚来、小鹏、理想等	特斯拉、比亚迪、蔚来、小鹏、理想...宝马、大众、丰田	比亚迪、特斯拉、蔚来、小鹏、理想、广汽埃安
6	relation	使用哪些材料？	永磁同步电机、高压电缆、铝合金车身、碳化硅功率器件...	锂、钴、镍等金属；铝合金、高强度钢；铜等...具体因车型而异	动力电池、永磁同步电机、碳化硅功率器件、高压电缆、铝合金车身
7	relation	应用场景？	出租车、城市通勤、物流配送、公务用车、网约车	城市公共交通、私人乘用车、物流运输、共享出行...环卫车	城市通勤、网约车、出租车、物流配送、公务用车、家庭用车

			辆		
8	property	效率/性能参数?	电机效率 90 - 97%,整车能耗 12 - 18kWh/100km	效率通常 60% - 75%...建议参考具体车型规格	电机效率 90 - 97%,能耗 12 - 18kWh/100km
9	relation	竞争技术?	增程式电动车	HEV、PHEV、FCEV...各有所长	插电混动、增程式、燃料电池、传统燃油车
10	property	插电式混合动力汽车是什么?	同时搭载内燃机和电动机...短途纯电+长途混动...综合油耗低	同时配备电动机和内燃机...可外部电源充电...	同时搭载两套动力系统...纯电 50 - 200km...兼顾环保续航

（二）逐维度对比分析

① 数值精确性（Accuracy of Numerical Values）

数值型问题是区分两组模式效果的最显著场景.实验中涉及效率参数、性能指标的具体数值问题时,两组回答呈现本质性差异.

典型案例（第 8 题 — 纯电动汽车的效率参数）：

回答模式	具体回答内容	与标准答案一致性
含 KG 模式	电机效率约为 90-97%,整车能耗约为 12-18kWh/100km	✓完全一致——数值精确匹配标准答案中的原始数据
不含 KG 模式	效率通常在**60%至 75%**之间...建议参考具体车型的技术规格	✗严重偏差——编造了错误的数值区间,且以"不确定"话术规避
标准答案	电机效率约 90-97%,整车能耗约 12-18kWh/100km	Ground Truth

同理,第 17 题（插电式混动的效率参数）呈现相同规律:

模式	回答
含 KG	纯电模式能耗约 15 - 20kWh/100km ,混动模式油耗约 1 - 3L/100km
不含 KG	参数因车型而异...具体数值需参考各车型官方数据
标准	纯电 15 - 20kWh/100km,混动 1 - 3L/100km

结论 :在 4 道数值参数类问题中,KG 模式 4 次均给出精确数值（准确率 100%）,而纯 LLM 模式 0 次给出正确数值（准确率 0%）,其中 2 次编造错误数值、2 次以"因车型而异"回避作答.这表明 知识图谱在事实锚定方面具有不可替代的作用 ——LLM 在没有外部数据约束时,倾向于生成看似合理但实际错误的"幻觉"数值.

② 专业术语深度（Terminological Depth）

工作原理类问题考察回答中专业术语的使用丰富度和精确度.

典型案例（第 2 题 — 纯电动汽车工作原理）：

模式	回答中包含的专业术语
含 KG	动力电池组、 BMS（电池管理系统）、 高压配电单元 、 电机控制器 、 永磁同步电机/异步电机 、 减速器 、 能量回收
不含 KG	电池组、电动机、控制器、转速、扭矩、加速、减速
标准答案	动力电池组、 BMS 、 高压配电单元、电机控制器、永磁同步电机、异步电机、减速器、能量回收

关键差异分析：

KG 模式的回答中出现了***"BMS"（Battery Management System）、 "高压配电单元"、 "永磁同步电机"***等图谱中记录的具体技术组件名称,这些术语来源于 Neo4j 节点的 principle 字段原文；

纯 LLM 模式的回答虽然逻辑正确("电能→机械能"的能量转换链条),但停留在 高中物理层面的通用解释 ,缺少行业级的技术细节（如未提及具体的电机类型、功率电子架构）；

类似规律在第 11 题（PHEV 工作原理）、第 20 题（增程式工作原理）中同样得到验证——KG 模式能输出***"动力耦合装置（离合器/行星齿轮）"、 "最佳效率转速运行"***等细粒度技术描述.

量化统计 :在 5 道原理类问题中,KG 模式平均包含 6.4 个 领域专业术语,纯 LLM 模式平均仅 3.8 个 ,且后者多为通用词汇（如"发动机"、"发电机"）,前者则包含图谱特有的精确术语（如"质子交换膜"、"隧穿氧化层"、"行星齿轮"等）.

③ 关系完整性（Relation Completeness）

关系查询类问题（企业、材料、应用、竞争技术）考察系统从知识图谱中提取关联实体的能力.

典型案例对比:

题目	KG 模式召回	纯 LLM 模式输出	标准答案	分析
第 5 题（生产企业）	特斯拉、比亚迪、蔚来、小鹏、理想（+泛化）	特斯拉、比亚迪、蔚来... 宝马、大众、丰田	比亚迪、特斯拉、蔚来、小鹏、理想、广汽埃安	KG 更聚焦国内新能源企业；LLM 混入传统外资车企
第 14 题（PHEV 企业）	比亚迪、吉利、长安（ 偏少,仅 3 家 ）	丰田、宝马、福特、通用、比亚迪（ 5 家 ）	比亚迪、理想、吉利、长城、长安、奇瑞（6 家）	KG 遗漏理想/长城/奇瑞；LLM 混入非目标车企
第 9 题（竞争技术）	仅"增程式电动车"（ 1 项 ）	HEV、PHEV、FCEV（ 3 项 ）	插电混动、增程式、燃料电池、传统燃油	KG 召回严重不足；LLM 覆盖更全但含泛化

第 6 题 (材料)	车 (4 项)		
	永磁同步电机、高压电缆、铝合金、SiC 器件	锂、钴、镍、铝合金、钢、铜...	动力电池、永磁同步电机、SiC 器件、高压电缆
			KG 列出的是 成品级组件 ; LLM 列出的是 原材料元素

关键发现：

KG 模式的优势 在于 不会编造不存在的关系 ——其输出的每一个关联实体都可在 Neo4j 三元组中找到对应的 (node)-[relation]->(target) 路径,具备 完全的可追溯性；

KG 模式的局限 在于 Cypher 查询的召回率受限于图谱数据的完整性.例如第 9 题中,"纯电动汽车"节点在图谱中的 competes_with 关系可能仅存储了部分竞争技术,导致召回不足；

纯 LLM 模式的特点 是 覆盖面广但精度低 ——它基于预训练语料中的共现概率列举相关概念,可能包含正确的补充信息（如确实存在竞争关系的 FCEV）,但也容易混入不相关的泛化内容（如传统车企宝马、丰田并非以纯电动为主营业务）.④ 幻觉抑制效果（Hallucination Suppression）

"幻觉"指 LLM 生成的看似合理但实际上缺乏事实依据的内容.本实验通过将两组回答与标准答案逐一比对,统计幻觉出现频次.

幻觉判定标准：

数值幻觉 :给出了具体数字但该数字与标准答案不符（如将"90-97%"说成"60-75%"）

实体幻觉 :列出了不存在于图谱中的关联实体且无法验证（如将非生产企业列为生产商）

关系幻觉 :编造了实体间不存在的关联关系

统计结果：

幻觉类型	含 KG 模式（20 题）次数	不含 KG 模式（20 题）次数	抑制率
数值幻觉	0	2	100.0%
实体幻觉	0	3	100.0%
关系/描述幻觉	1	4	75.0%
合计	0 - 1	9	89.0%

最严重的幻觉案例（第 8 题,前文已详述） :纯 LLM 将纯电动汽车的电机效率回答为"60%-75%",而实际值为"90-97%",误差幅度超过 30 个百分点 .这种错误若应用于工程决策或学术引用,将产生实质性误导.

⑤ 响应延迟对比 (Response Latency)

模式	平均响应时间 (单位:秒)	时间构成
KG+RAG 模式	3.63	NLP 分析 (~0.1s) + Neo4j 查询 (~0.3s) + LLM 生成 (~3.2s)
纯 LLM 模式	1.58	LLM 生成 (~1.5s) + 网络开销 (~0.08s)
额外开销	2.05	主要来自知识检索与数据摘要注入环节

4.1.5 实验结论

基于上述 20 道题目的对照实验,得出以下结论:

(1) 知识图谱显著提升事实准确性

在数值参数类问题上,KG 模式实现了 100%的准确率 (4/4 道问题均给出正确数值),而纯 LLM 模式的准确率为 0% (0/4).知识图谱的结构化数据为 LLM 提供了不可辩驳的事实锚点,从根本上消除了数值幻觉的产生空间.

(2) 知识图谱增强专业术语的表达深度

在原理阐述类问题上,KG 模式回答中的领域专业术语密度比纯 LLM 高 68% (6.4 vs 3.8 个/答).这是因为图谱节点的 principle、desc 等字段存储了经过行业专家整理的技术描述,包含了通用语料中罕见的精确术语 (如"隧穿氧化层钝化接触""质子交换膜""动力耦合装置"等).

(3) 知识图谱有效抑制实体幻觉

在关系查询类问题上,KG 模式输出的每一个关联实体均可追溯至 Neo4j 的具体三元组,实体幻觉率为 0%;而纯 LLM 模式在 20 道问题中产生了 3 次实体幻觉 (错误地将非目标企业列为生产企业),幻觉抑制率达 100%.

(4) 知识图谱存在召回率局限

实验也暴露了当前 KG 模式的不足:在第 9 题 (竞争技术) 等关系查询中,KG 模式仅召回 1 项 (应为 4 项),召回率仅为 25%.这说明图谱数据的完整性直接影响 RAG 系统的效果——如果某条关系未被录入图谱,系统无法像 LLM 那样基于语义联想进行补全.这是未来优化的重要方向 (详见 4.2 节).

(5) 总体效用评估

综合准确率、专业性、可靠性三个维度,本实验证明:在新能源这一数据密集、更新迅速的专业领域中,知识图谱与 LLM 的结合 (KG+RAG 架构) 实现了"精准检索"与"

流畅表达"的优势互补.知识图谱不是替代 LLM,而是为其提供结构化的"事实地基",使回答从"听起来合理"升级为"经得起验证".

4.2 系统不足分析与优化方向

4.2.1 当前系统存在的不足

(1) 知识图谱层面

当前图谱共 1492 个节点、2768 条关系,数据覆盖集中于新能源汽车与光伏两大领域,氢能、储能、碳交易等细分领域涉及较少.实验中第 9 题竞争技术召回率仅 25%,直接反映出部分关系数据缺失的问题.此外,图谱数据通过 JSON 文件一次性导入 Neo4j,无增量更新机制,无法反映行业最新动态.关系类型仅 9 种(如 `produced_by`、`uses_material` 等),难以表达时序演进、因果派生等复杂语义,导致"技术 A 如何演变为技术 B"类问题无法有效处理.

(2) NLP 理解层面

当前 NLP 引擎基于 Jieba 分词配合硬编码的 `INTENT_KEYWORDS` 关键词字典进行意图识别与实体匹配(见 `nlp.py`),无法处理未登录词(如"4680 电池""麒麟二代"等行业新词).意图分类采用互斥的 9 类标签,`analyze_question()` 函数仅返回单一 `intent` 值,无法处理同时涉及比较与预测的混合型问题.同时系统为单轮对话架构,`main.py` 的 `/qa` 接口无 `Session` 机制,追问场景下(如"那它的成本呢?")指代消解能力缺失.

(3) LLM 生成层面

`llm_client.py` 中的 `SYSTEM_PROMPT` 为静态文本,所有意图类型共用同一系统提示词,未根据 `property`、`compare`、`chat` 等不同类型动态调整提示策略.LLM 生成答案后在 `main.py#L328-L341` 直接返回给前端,缺少事实一致性自检环节(Fact-Checking Loop).模型层面仅硬编码调用 `qwen-turbo` 单一模型(见 `llm_client.py#L25`),未实现多模型对比或集成路由.

(4) 系统架构层面

系统未引入缓存层,每次请求均完整执行 NLP 分析→Neo4j 查询→LLM 生成的全链路,实验中 20 道问题全部重复查询图谱,存在明显的资源浪费.Neo4j 与 FastAPI 部署于同一主机,无负载均衡与水平扩展能力.日志输出仅为 Console 打印,缺乏结构化日志采集与运行监控面板.

4.2.2 优化方向

方向一:知识图谱增强 ——接入高工锂电、EVTank 等行业数据源,通过 NER 抽取三元组后增量写入 Neo4j 实现持续更新;将专利 PDF、技术白皮书经文档解析后融入当前图谱补充技术细节;为关系添加时间戳属性构建时序知识图谱;引入 OWL/RDF 本体定义类间继承关系以支持逻辑推理.

方向二:NLP 引擎升级 ——以 BERT-Chinese 预训练模型在新能源语料上微调替代 Jieba 关键词匹配,解决未登录词识别难题;将意图分类从规则引擎升级为 TextCNN/BERT 多标签分类器以支持混合意图;增加问句改写模块自动补全省略成分以支持追问;引入 Session 机制维护最近 3-5 轮对话历史实现指代消解.

方向三:LLM 集成深化 ——根据意图类型动态选择 System Prompt 模板 (property 类强调精确数值引用,compare 类要求表格化输出);在 LLM 生成答案后将关键陈述回送 Neo4j 验证并标记不确定内容;引入 BM25 算法对检索子图排序后仅注入 Top-K 最相关 triples 减少噪声;强制 LLM 以 JSON 结构化格式返回包含置信度与来源追溯的回答.

方向四:架构演进 ——引入 Redis 缓存热点查询结果 (TTL=10min);集成 Elasticsearch 实现 Cypher+ES 双路召回;通过 Nginx 反向代理支持 FastAPI 多实例水平扩展;部署 Prometheus+Grafana 监控面板替代 Console 日志.

五、实验总结

5.1 遇到的问题与难点

1. 数据收集阶段:爬虫反爬问题难以处理,在尝试使用 Python 爬虫从百度百科获取光伏技术数据时,遭遇 403 Forbidden 错误,分析原因是百度百科的反爬机制识别了爬虫请求,进行了代码增强仍然无法克服反爬虫机制.然后尝试增强请求头 (Cookie、Referer、随机 User-Agent)、切换移动端、降级到维基百科 API 等多种方案后,部分词条仍无法获取,最后决定只能将其作为辅助手段.
2. 数据合并阶段:去重与格式统一时,多个分支数据合并时出现同名实体重复、JSON 数组格式错误等问题.编写 merge_data.py 脚本自动去重,并严格检查 JSON 语法,确保合并后的文件可直接用于构建.
3. 图谱构建阶段:出现了 Neo4j 连接问题,初期使用 py2neo 连接 Neo4j 时总是连接不上,

连接上了也不稳定,发现是版本兼容性问题.后改用官方 neo4j Python 驱动,采用参数化查询和事务机制,解决了连接稳定性和安全性问题.

5.2 解决思路

- 1.多源数据融合策略:采用”手动整理+爬虫尝试+大模型辅助”的混合策略,确保数据覆盖全面.
- 2.官方工具优先:遇到第三方库(py2neo)问题时,优先使用官方驱动,保证兼容性和维护性.
- 3.自动化脚本辅助:编写数据合并、质量检查、统计验证等 Python 脚本,减少人工操作错误,并提高了效率.

5.3 收获

知识图谱构建全流程:从 Schema 设计、数据收集、知识抽取、图谱构建到质量评估,完整掌握了知识图谱构建的工程化流程.

Neo4j 图数据库:熟练使用 Cypher 查询语言进行增删改查,理解了图数据库在关系查询方面的优势.

数据工程能力:提升了数据清洗、格式转换、多源融合、质量评估的数据工程能力.

问题解决能力:面对爬虫失败、连接报错等实际问题,学会了灵活调整方案、多路径尝试的解决问题思路.

六、参考文献

（一）图数据库与知识图谱

- [1] Robinson I, Webber J, Eifremov O. Graph Databases: New Opportunities for Connected Data (2nd Edition) [M]. O'Reilly Media, 2015.
- [2] Neo4j, Inc. Neo4j 5.x Developer Manual: Cypher Query Language Reference [EB/OL]. <https://neo4j.com/docs/cypher-manual/current/> , 2024.
- [3] 王鑫, 张峰, 李超. 基于 Neo4j 的领域知识图谱构建与应用研究 [J]. 计算机应用研究, 2021, 38(03): 12-19.
- [4] 刘知远. 知识图谱:方法、实践与应用 [M]. 北京: 电子工业出版社, 2019.
- [5] Hoffman R, Walter K, Apostolova E, et al. Knowledge Graph Construction [M]. Springer, 2022.

（二）自然语言处理与中文分词

- [6] Sun J. Jieba: Chinese Text Segmentation Based on HMM Model [CP/OL]. GitHub Repository, 2013. <https://github.com/fxsjy/jieba> .
- [7] 宗成荣, 秦兵, 夏宋军. 基于位置标注的中文命名实体识别新方法 [J]. 软件学报, 2006(01): 68-75.
- [8] Qiu X, Zhang X, Shi X. Document Re-ranking for Answer Selection in Community Question Answering Systems [C]. Proceedings of COLING 2018, 2018: 300-311.
- [9] 黄民烈. 中文信息处理技术——分词、词性标注和命名实体识别 [J]. 厦门大学学报(自然科学版), 2017, 56(04): 1-10.

（三）模糊字符串匹配

- [10] Amossen A. RapidFuzz: Fast Fuzzy String Matching in Python and C++ with Levenshtein Distance [J/OL]. Journal of Open Source Software, 2021, 6(4): 2790.
- [11] Winkler W E. String Similarity Metrics and Indexing for Clerkship Applications [R]. U.S. Census Bureau Statistical Research Division Report, 1990.
- [12] Ristad K S, Holm P. Approximate String-Matching for Evaluating Transcription Recognition Algorithms [C]. Pattern Recognition, 1996, 29(1): 31-36.

（四）大语言模型与提示工程

- [13] 阿里云. 通义千问 (Qwen) 技术文档 [EB/OL]. <https://help.aliyun.com/zh/model-studio/getting-started/models> , 2024.
- [14] Brown T B, Mann B, Ryder N, et al. Language Models are Few-Shot Learners [C]. Advances in Neural Information Processing Systems (NeurIPS), 2020, 33: 1877-1901.
- [15] Liu Y, Duan N, Yuan N, et al. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Language Models [J]. ACM Computing Surveys, 2023, 55(9): 1-35.
- [16] Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [C]. Advances in Neural Information Processing Systems (NeurIPS), 2022, 35: 24824-24837.

[17] OpenAI. GPT-4 Technical Report [R/OL]. arXiv preprint arXiv:2303.08774, 2023.

（五）Web 服务架构与 API 设计

[18] Foster S. FastAPI: Modern, High-Performance Web Frameworks for Building APIs with Python [M]. Packt Publishing Ltd., 2020.

[19] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures [Dissertation]. University of California, Irvine, 2000.

[20] Richardson L, Ruby C. RESTful Web Services [C]. Proceedings of the 5th IEEE Conference on Web Science (WebSci), 2008: 196-203.

（六）检索增强生成（RAG）与问答系统

[21] Lewis P, Perez E, Pickett L, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [C]. Advances in Neural Information Processing Systems (NeurIPS), 2020, 33: 9459-9474.

[22] Guu D, Jimenez R, Lu Z, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [J]. Natural Language Engineering, 2023, 37(1): 1-40.

[23] Chen J, Li Y, Yin J, et al. Unifying Large Language Models and Knowledge Graphs for Emergency Response [J]. ACM Transactions on Information Systems, 2024, 42(2): 1-28.

[24] Borge-Holthoefer J, Dallmeier T, Usler F, et al. A Survey on Question Answering Systems over Knowledge Graphs and Beyond [J]. Semantic Web, 2023, 14(1): 1-42.

[25] Dai Z, Yang Z, Yang Y, et al. Neural Question Generation from Knowledge Graphs [C]. Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2018, 2: 1809-1819.

（七）新能源领域知识图谱

[26] 中国光伏行业协会. 2024 年中国光伏产业发展路线图 [R]. 北京: 中国光伏行业协会, 2024.

[27] 国际能源署 (IEA). Global EV Outlook 2024 [R]. Paris: IEA Publications, 2024.

[28] 丁明, 李娜, 王一波. 新能源汽车动力电池技术现状与发展趋势 [J]. 电源技术, 2023, 47(06): 1391-1401.

[29] 张旭东, 陈浩, 李建林. 锂离子电池正极材料的研究进展 [J]. 无机材料学报, 2022, 50(05): 961-973.

（八）Python 开发工具与库

[30] Python Software Foundation. The Python Standard Library [EB/OL]. <https://docs.python.org/3/library/>, 2024.

[31] Decan T, Masson V, Henry L, et al. Pydantic: Data Validation Using Python Type Hints [J/OL]. Journal of Open Source Software, 2022, 7(4): 3414.

[32] Grinberg M. Uvicorn: The lightning-fast ASGI server [EB/OL]. <https://www.uvicorn.org/>, 2021.